SHOPWAVE

Professional E-Commerce MVP

Agile Project Documentation · Scrum Framework

Document Type	Project Charter + Product Backlog + Sprint Plans
Methodology	Scrum / Agile (2-Week Sprints)
Tech Stack	Angular · Node.js + Express · MongoDB
Total Story Points	~191 points across 39 User Stories

SECTION 1 — PROJECT CHARTER

1.1 Project Overview

Project Name	ShopWave — Professional E-Commerce MVP
Version	1.0
Date	March 26, 2026
Document Owner	Team Lead / Full-Stack Developer
Methodology	Scrum / Agile (2-week Sprints)
Tech Stack	Angular · Node.js + Express · MongoDB · JWT · Stripe / Paymob

1.2 Vision Statement

> *"To deliver a scalable, secure, and intuitive e-commerce platform that enables businesses to sell online effectively — built with best-practice Agile engineering and modern web technologies."*

1.3 Objectives

- **Technical:** Implement a production-grade REST API with JWT authentication, role-based access control, and full CRUD operations.
- **Functional:** Deliver a complete customer buying journey — browse → cart → checkout → payment — integrated with a real payment gateway.
- **Quality:** Achieve ≥ 60% unit test coverage; zero critical security vulnerabilities; full API documentation via Swagger.
- **Agile:** Follow the Scrum framework with documented sprints, a groomed backlog, velocity tracking, and retrospectives.
- **Academic:** Produce a graduation-level deliverable demonstrating mastery of Angular, Node.js, MongoDB, and Agile practices.

1.4 Stakeholders

Stakeholder	Type	Role / Interest	Influence
Product Owner / Supervisor	Academic	Approves scope & deliverables	High
Team Lead (Full-Stack Dev)	Internal	Architecture, code, delivery	High
Frontend Developer(s)	Internal	Angular UI implementation	High
Backend Developer(s)	Internal	API & database implementation	High
End Customers	External	Browse, search & purchase products	High

Admin Users	External	Manage products, orders & users	Medium
Payment Gateway (Stripe/Paymob)	Third-Party	Process transactions securely	Medium

1.5 Assumptions & Constraints

Assumptions

- Team has working knowledge of Angular, Node.js, and MongoDB.
- Payment gateway will be used in sandbox/test mode only.
- Cloud storage (Cloudinary) will handle product image uploads.
- A free-tier MongoDB Atlas cluster is sufficient for development.

Constraints

- Academic deadline defines the final release date (Sprint 6).
- Team size: 2–4 developers; no dedicated QA or DevOps resource.
- Budget: zero — all tools and services must use free tiers.

1.6 Success Criteria

ID	Category	Acceptance Criterion
SC-01	**Authentication**	JWT login/register with role-based access (Admin & Customer) fully functional
SC-02	**Product Catalog**	Admin can CRUD products; customers can browse, filter, and search
SC-03	**Shopping Cart**	Add/remove/update cart items with real-time totals
SC-04	**Checkout & Payment**	End-to-end checkout integrated with Stripe or Paymob (test mode)
SC-05	**Order Management**	Customers can view order history; Admins can update order status
SC-06	**Performance**	API response time < 500ms for 95% of requests under normal load
SC-07	**Security**	No critical OWASP Top 10 vulnerabilities; all endpoints authenticated
SC-08	**Code Quality**	Unit test coverage ≥ 60%; CI pipeline passes on every merge
SC-09	**Documentation**	Swagger/OpenAPI docs for all API endpoints; README up to date
SC-10	**Deployment**	App deployed to a cloud environment (Vercel / Railway / Render)

SECTION 2 — PRODUCT BACKLOG

2.1 Epics

- **EPIC-01 Authentication & User Management:** Registration, login, JWT, OAuth, profile.
- **EPIC-02 Product Catalog:** CRUD, search, filter, pagination, product detail.
- **EPIC-03 Shopping Cart:** Add/update/remove items, persist cart, totals.
- **EPIC-04 Checkout & Payment:** Shipping, payment gateway, confirmation email.
- **EPIC-05 Order Management:** Order history, status tracking, cancellation.
- **EPIC-06 Admin Dashboard:** KPI overview, user management, reports.
- **EPIC-07 Reviews & Ratings:** Customer reviews, moderation.
- **EPIC-08 Notifications:** Email alerts for order status and low stock.

2.2 User Story Format

Format: *"As a [User Role], I want to [Action/Goal], so that [Business Value]."*

Story Points: Fibonacci scale — 1 | 2 | 3 | 5 | 8 | 13 | 21

2.3 Full Product Backlog

ID	Epic	User Story	MoSCoW	Priority	Points
US-001	Auth	As a new user, I want to register with email & password, so that I can create an account and start shopping.	Must-Have	Critical	8
US-002	Auth	As a registered user, I want to log in securely using JWT, so that my session is protected and persistent.	Must-Have	Critical	5
US-003	Auth	As a logged-in user, I want to log out, so that I can end my session securely.	Must-Have	Critical	2
US-004	Auth	As a user, I want to view and edit my profile (name, address, phone), so that my personal info is up to date.	Should-Have	High	5
US-005	Auth	As a user, I want to reset my password via email link, so that I can recover my account if I forget my password.	Should-Have	Medium	5
US-006	Auth	As a user, I want to log in via Google OAuth, so that I can sign in faster without a separate password.	Could-Have	Low	8
US-007	Catalog	As an Admin, I want to add a new product (name, description, price, images, stock), so that customers can find and buy it.	Must-Have	Critical	8
US-008	Catalog	As an Admin, I want to edit an existing product, so that I can keep product info accurate.	Must-Have	Critical	5
US-009	Catalog	As an Admin, I want to delete a product, so that discontinued items are removed from the store.	Must-Have	High	3

US-010	Catalog	As a Customer, I want to browse all products in a paginated catalog, so that I can discover items without scrolling endlessly.	Must-Have	Critical	5
US-011	Catalog	As a Customer, I want to search products by keyword, so that I can quickly find what I am looking for.	Must-Have	Critical	5
US-012	Catalog	As a Customer, I want to filter products by category and price range, so that I can narrow down my choices.	Must-Have	High	5
US-013	Catalog	As a Customer, I want to view a product detail page with images, description, and reviews, so that I can make an informed purchase decision.	Must-Have	Critical	5
US-014	Catalog	As a Customer, I want to see a "Related Products" section on the detail page, so that I can discover similar items.	Should-Have	Medium	3
US-015	Catalog	As a Customer, I want to add a product to my Wishlist, so that I can save it for later consideration.	Could-Have	Low	5
US-016	Cart	As a Customer, I want to add products to my cart, so that I can collect items before purchasing.	Must-Have	Critical	5
US-017	Cart	As a Customer, I want to update item quantities in my cart, so that I can buy the right amount.	Must-Have	Critical	3
US-018	Cart	As a Customer, I want to remove items from my cart, so that I can change my mind before checkout.	Must-Have	Critical	2
US-019	Cart	As a Customer, I want to see the cart total (subtotal, taxes, shipping), so that I know the exact cost before paying.	Must-Have	High	3
US-020	Cart	As a Customer, I want my cart to persist across sessions, so that I do not lose items when I close the browser.	Should-Have	High	5
US-021	Cart	As a Customer, I want to apply a discount coupon code at checkout, so that I can benefit from promotional offers.	Could-Have	Medium	8
US-022	Checkout	As a Customer, I want to enter a shipping address at checkout, so that my order is delivered to the correct location.	Must-Have	Critical	3
US-023	Checkout	As a Customer, I want to pay securely via Stripe or Paymob, so that my transaction is processed safely.	Must-Have	Critical	13
US-024	Checkout	As a Customer, I want to receive an order confirmation email after purchase, so that I have proof of my order.	Must-Have	High	5
US-025	Checkout	As a Customer, I want to select a shipping method (standard / express), so that I can choose delivery speed vs. cost.	Should-Have	Medium	3
US-026	Checkout	As a Customer, I want to save my card details for future payments, so that checkout is faster next time.	Could-Have	Low	5
US-027	Orders	As a Customer, I want to view my order history with statuses, so that I can track all my purchases.	Must-Have	High	5
US-028	Orders	As a Customer, I want to view full order details (items, costs, address), so that I have a clear record.	Must-Have	High	3

US-029	Orders	As an Admin, I want to view all customer orders in a dashboard, so that I can manage fulfillment.	Must-Have	Critical	5
US-030	Orders	As an Admin, I want to update order status (Pending → Processing → Shipped → Delivered), so that customers are kept informed.	Must-Have	Critical	5
US-031	Orders	As a Customer, I want to cancel an order (while still Pending), so that I can change my mind before shipment.	Should-Have	Medium	5
US-032	Orders	As a Customer, I want to request a return/refund, so that I can get my money back for faulty items.	Could-Have	Low	8
US-033	Admin	As an Admin, I want a dashboard with KPI widgets (revenue, orders, new users), so that I have a real-time business overview.	Should-Have	High	8
US-034	Admin	As an Admin, I want to manage user accounts (view, deactivate), so that I can handle account issues.	Should-Have	Medium	5
US-035	Admin	As an Admin, I want to export order reports as CSV, so that I can analyze data in external tools.	Could-Have	Low	5
US-036	Reviews	As a Customer, I want to leave a star rating and review on a product I purchased, so that other shoppers benefit from my experience.	Should-Have	Medium	5
US-037	Reviews	As an Admin, I want to moderate and delete inappropriate reviews, so that the platform remains trustworthy.	Should-Have	Low	3
US-038	Notif.	As a Customer, I want to receive an email when my order status changes, so that I stay informed without logging in.	Should-Have	Medium	5
US-039	Notif.	As an Admin, I want to receive a notification when stock falls below a threshold, so that I can restock in time.	Could-Have	Low	5

SECTION 3 — MoSCoW PRIORITISATION

3.1 Prioritisation Summary

Priority Tier	Description	# Stories	Story Points	Examples
Must-Have	MVP Core	22	~105	Auth, Cart, Payment, Orders
Should-Have	Quality Add-ons	11	~52	Dashboard, Reviews, Emails
Could-Have	Nice to Have	6	~34	OAuth, Wishlist, Coupons
Won't Have (v1)	Future Scope	-	-	Mobile App, Multi-vendor

3.2 Won't Have (v1.0 Scope Exclusions)

The following items are explicitly out of scope for this release. They are documented here to manage stakeholder expectations and may be revisited in a future product roadmap.

- Native Mobile Application (iOS / Android)
- Multi-vendor / Marketplace functionality
- Advanced analytics with BI dashboards
- Live chat / chatbot support
- Multi-language & currency support (internationalisation)
- Subscription / recurring payment plans

3.3 Prioritisation Rationale

Must-Have items form the irreducible core of a functional e-commerce platform. Without authentication, a product catalog, a working cart, and a payment gateway, the product has no value to any user.

Should-Have items significantly improve the user experience and admin efficiency. While the MVP works without them, they are planned for Sprint 3–4 as velocity allows.

Could-Have items are desirable enhancements (coupons, wishlist, OAuth) that require more effort with lower immediate business impact. They are deferred to Sprint 5 or a v1.1 release.

SECTION 4 — SPRINT 0 & SPRINT 1 PLANS

4.1 Sprint 0 — Initialisation & Setup

Sprint	Duration	Goal	DoD
Sprint 0	2 Weeks	All repos, pipelines, and schemas ready to code	Dev env set up on all machines; CI passing; ERD reviewed

Sprint 0 Task Breakdown

Task ID	Category	Task Description	Owner	Estimate
SP0-01	**Project Setup**	Initialize Angular workspace with routing, lazy loading, and standalone components	Dev	4h
SP0-02	**Project Setup**	Scaffold Node.js + Express backend with folder structure (routes/controllers/services/models)	Dev	3h
SP0-03	**Project Setup**	Configure MongoDB Atlas cluster & Mongoose connection with .env management	Dev	2h
SP0-04	**Project Setup**	Set up Git repository, branching strategy (main/develop/feature/*), and PR templates	DevOps	2h
SP0-05	**CI/CD**	Configure GitHub Actions pipeline: lint → test → build on PR	DevOps	4h
SP0-06	**Design**	Finalise UI design system: color palette, typography, component library (Angular Material)	Design	4h
SP0-07	**Architecture**	Define API contract: RESTful endpoints, naming conventions, error response schema	Lead	3h
SP0-08	**Architecture**	Create Swagger/OpenAPI base file and integrate swagger-ui-express middleware	Dev	3h
SP0-09	**Data Model**	Design & document MongoDB schemas: User, Product, Category, Cart, Order, Review	Dev	4h
SP0-10	**Planning**	Create Jira/Trello board, import backlog, assign story points, and agree on Definition of Done	PO	2h
SP0-11	**Environment**	Set up .env templates for dev/staging/prod, configure ESLint + Prettier for both repos	Dev	2h
SP0-12	**Security**	Research & document JWT strategy, CORS policy, Helmet.js setup, and rate-limiting plan	Lead	3h

4.2 Sprint 1 — Authentication, Product Catalog & Cart Core

Sprint	Duration	Goal	Velocity	Stories Covered	Points

Sprint 1	2 Weeks	Users can register, log in, browse products, and add items to cart	~64 pts	US-001–003, US-007–013, US-016–019		~64

Sprint 1 Task Breakdown

Task ID	Category	Task Description	Stories	Owner	Points	Estimate
SP1-01	**Auth**	Build User model (schema + validation) and Auth service (register/login)	US-001, 002	Backend	8	5h
SP1-02	**Auth**	Implement JWT issuance, refresh-token strategy, and Auth middleware	US-002, 003	Backend	5	3h
SP1-03	**Auth**	Create role-based access guard (isAdmin / isCustomer) and attach to routes	US-002, 003	Backend	3	2h
SP1-04	**Auth**	Build Angular Auth module: register & login forms with reactive validation	US-001, 002	Frontend	8	2h
SP1-05	**Auth**	Implement AuthGuard & RoleGuard, intercept JWT in HTTP requests	US-002, 003	Frontend	5	4h
SP1-06	**Catalog**	Implement Product & Category Mongoose models with indexes	US-007, 008	Backend	5	4h
SP1-07	**Catalog**	Build CRUD product API endpoints with image upload (Multer + Cloudinary)	US-007-009	Backend	8	3h
SP1-08	**Catalog**	Add product search (text index), filter (category/price), and pagination	US-010-012	Backend	8	3h
SP1-09	**Catalog**	Build product catalog page with Angular grid, search bar, filter sidebar	US-010-012	Frontend	8	4h
SP1-10	**Catalog**	Build product detail page with image gallery, description, and Add-to-Cart CTA	US-013	Frontend	5	2h
SP1-11	**Cart**	Implement Cart model & API (add/update/remove/get by userId)	US-016-019	Backend	5	2h
SP1-12	**Cart**	Build cart drawer/page in Angular with live quantity controls and total calculation	US-016-019	Frontend	8	3h
SP1-13	**Testing**	Write unit tests for Auth service (register, login, token validation) — Jest	US-001, 002	Backend	3	5h
SP1-14	**Testing**	Write unit tests for Product service (CRUD, search, filter) — Jest	US-007-012	Backend	3	6h
SP1-15	**Docs**	Update Swagger docs for all Auth, Product, and Cart endpoints	All	Lead	2	4h

4.3 Scrum Ceremonies Schedule

Ceremony	Frequency	Duration	Purpose
Sprint Planning	Start of Sprint	2 hours	Decompose stories into tasks; agree on sprint goal and capacity
Daily Stand-up	Every weekday	15 mins	What did I do? What will I do? Any blockers?
Sprint Review	End of Sprint	1 hour	Demo working software to Product Owner; collect feedback
Sprint Retrospective	End of Sprint	45 mins	What went well / could improve / action items for next sprint
Backlog Refinement	Mid-Sprint	1 hour	Estimate, split, and clarify upcoming stories

4.4 Definition of Done (DoD)

Category	Done Criterion
Code	Feature branch merged to develop via approved Pull Request
Code	No linting errors (ESLint) or TypeScript compilation warnings
Testing	Unit tests written and passing (coverage ≥ 60% for new code)
Testing	Manual QA performed in a local or staging environment
API	Swagger documentation updated for any new/modified endpoints
Review	Peer code review completed by at least one other developer
Acceptance	Product Owner has accepted the story against acceptance criteria
Deploy	Feature tested in staging environment without regression failures

SECTION 5 — RELEASE ROADMAP & VELOCITY

5.1 Sprint Timeline Overview

Sprint	Duration	Theme	Stories / Tasks	Story Points
Sprint 0	2 weeks	Initialization	Repos, CI/CD, DB, schema, design system, API contract	—
Sprint 1	2 weeks	Auth + Catalog + Cart Core	US-001–003, US-007–013, US-016–019	~64
Sprint 2	2 weeks	Checkout + Payment	US-022–024, US-027–030	~34
Sprint 3	2 weeks	Admin Dashboard + Orders	US-029–031, US-033–034	~26
Sprint 4	2 weeks	Should-Have Polishing	US-004,005,014,020,025,036–038	~34
Sprint 5	2 weeks	Could-Have + Bug-Fix	US-006,015,021,032,035,039	~28
Sprint 6	1 week	Hardening & Launch	Performance, security audit, final docs, cloud deploy	—

5.2 Risk Register

#	Risk	Likelihood	Impact	Mitigation Strategy
R-01	Payment gateway integration delays	Medium	High	Spike in Sprint 1; use Stripe test keys early; fallback to mock payment service for demo
R-02	Scope creep from adding non-MVP features	High	High	Strict MoSCoW adherence; PO must approve any story promotions
R-03	Team member unavailability	Medium	Medium	Cross-functional pairing; document all setup steps; no knowledge silos
R-04	MongoDB performance bottlenecks	Low	Medium	Add indexes at schema design; monitor with Atlas performance advisor
R-05	JWT security vulnerability	Low	High	Short token expiry + refresh tokens; use Helmet.js; regular dependency audit

5.3 Technology Architecture Summary

Layer	Technology	Key Libraries / Tools

Frontend	Angular 21+	Angular Material, RxJS, NgRx (state), Angular Router, Reactive Forms
Backend	Node.js + Express	Mongoose, JWT (jsonwebtoken), bcryptjs, Multer, Nodemailer, Helmet, cors, express-rate-limit
Database	MongoDB Atlas	Mongoose ODM, text indexes, aggregation pipeline
Auth	JWT + bcryptjs	Access token (15m) + Refresh token (7d) strategy
Storage	Cloudinary	Product image upload via Multer middleware
Payments	Stripe / Paymob	Client-side Elements + server-side intent creation
CI/CD	GitHub Actions	Lint → Test → Build pipeline on every PR to develop
Hosting	Vercel + Railway	Angular SPA on Vercel; Express API on Railway / Render
Docs	Swagger / OpenAPI	swagger-ui-express + swagger-jsdoc
Testing	Jest + Supertest	Unit tests (services), integration tests (API routes)

Document prepared for academic graduation project submission. All stories and tasks are Jira/Trello-ready.